EXPT NO: 3	A PYTHON PROGRAM TO PERFORM CRITICAL PATH METHOD ANALYSIS
DATE:	

AIM:

To Perform Critical Path Method (CPM) Analysis For Project Scheduling And Management Using Python.

PROCEDURE:

1. INSTALL REQUIRED LIBRARIES:

    - Make sure you have pandas, networkx, and matplotlib installed. You can
    - install them using pip if not already installed.

        -bash

        -Copy code

        -pip install pandas networkx matplotlib

2. DEFINE THE PROBLEM:

    - Identify the tasks, their durations, and dependencies.

3. SET UP THE PROBLEM IN PYTHON:

    - Use NetworkX to represent the project tasks and their dependencies.
    - Use the topological sort to identify the critical path.

4. RUN THE CODE:

    - Execute the Python code to get the critical path and project duration.

5. ANALYZE THE OUTPUT:

    - Interpret the results and validate the solution against the project schedule.

PROBLEM 1 : SOLVE THE PROBLEM USING CPM

Task	A	B	C	D	E	F	G
Duration	3	2	4	2	3	1	2
Immediate Predecessor	-	A	A	B, C	C	D, E	F

CODE

import networkx as nx import matplotlib.pyplot as plt

class Task: def **init**(self, name, duration): self.name = name self.duration = duration self.early_start = 0 self.early_finish = 0 self.late_start = float('inf') self.late_finish = float('inf') self.successors = []

class CPM: def **init**(self, tasks, dependencies): self.tasks = {name: Task(name, duration) for name, duration in tasks.items()} self.dependencies = dependencies self.build_graph()

```
def build_graph(self):
    for task, deps in self.dependencies.items():
     for dep in deps:
            self.tasks[dep].successors.append(self.tasks[task])

def forward_pass(self):
    # Initialize start nodes
    for task in self.tasks.values():
        if not self.dependencies[task.name]:
            task.early_start = 0
            task.early_finish = task.duration

    # Forward calculation
    for task in sorted(self.tasks.values(), key=lambda t: t.early_start):
        for succ in task.successors:
            succ.early_start = max(succ.early_start, task.early_finish)
            succ.early_finish = succ.early_start + succ.duration

def backward_pass(self):
    max_finish = max(task.early_finish for task in self.tasks.values())

    # Initialize end nodes
    for task in self.tasks.values():
        if not task.successors:
            task.late_finish = max_finish
```

```
            task.late_start = task.late_finish - task.duration

    # Backward calculation
    for task in sorted(self.tasks.values(), key=lambda t: -t.early_finish):
        for succ in task.successors:
            task.late_finish = min(task.late_finish, succ.late_start)
            task.late_start = task.late_finish - task.duration

def find_critical_path(self):
    return [task.name for task in self.tasks.values()
            if task.early_start == task.late_start]

def run(self):
    self.forward_pass()
    self.backward_pass()
    return self.find_critical_path()

def visualize_network(self):
    G = nx.DiGraph()

    for task in self.tasks.values():
        label = f"{task.name}\nES:{task.early_start} EF:{task.early_finish}\nLS:{task.late_start}
LF:{task.late_finish}"
        G.add_node(task.name, label=label)

    for task in self.tasks.values():
        for succ in task.successors:
            G.add_edge(task.name, succ.name)

    pos = nx.spring_layout(G)
    labels = nx.get_node_attributes(G, 'label')

    plt.figure(figsize=(10, 6))
    nx.draw(G, pos, with_labels=True, node_size=3000,
            node_color="lightblue", edge_color="gray", font_size=10)
    nx.draw_networkx_labels(G, pos, labels, font_size=8)

    plt.title("Project Network Diagram")
    plt.show()
```

```
if name == "main": tasks = {'A': 3, 'B': 2, 'C': 4, 'D': 2, 'E': 3, 'F': 1, 'G': 2}

dependencies = {
    'A': [],
    'B': ['A'],
    'C': ['A'],
    'D': ['B', 'C'],
    'E': ['C'],
    'F': ['D', 'E'],
    'G': ['F']
}

cpm = CPM(tasks, dependencies)

critical_path = cpm.run()

print("Critical Path:", " -> ".join(critical_path))
```

OUTPUT

Critical Path: A -> C -> E -> F -> G

PROBLEM 2: SOLVE THE PROBLEM USING CPM

Tasks (Duration):

A = 2, B = 3, C = 2, D = 4, E = 3, F = 2

Dependencies:

A → Start

B → A

C → A

D → B

E → B, C

F → D, E

CODE

```
import networkx as nx import matplotlib.pyplot as plt
```

class Task: def **init**(self, name, duration): self.name = name self.duration = duration self.early_start = 0 self.early_finish = 0 self.late_start = float('inf') self.late_finish = float('inf') self.successors = []

class CPM: def **init**(self, tasks, dependencies): self.tasks = {name: Task(name, duration) for name, duration in tasks.items()} self.dependencies = dependencies self.build_graph()

```
def build_graph(self):
    for task, deps in self.dependencies.items():
        for dep in deps:
            self.tasks[dep].successors.append(self.tasks[task])

def forward_pass(self):
    for task in self.tasks.values():
        if not self.dependencies[task.name]:
            task.early_start = 0
            task.early_finish = task.duration

    for task in sorted(self.tasks.values(), key=lambda t: t.early_start):
        for succ in task.successors:
            succ.early_start = max(succ.early_start, task.early_finish)
            succ.early_finish = succ.early_start + succ.duration

def backward_pass(self):
    max_finish = max(task.early_finish for task in self.tasks.values())

    for task in self.tasks.values():
        if not task.successors:
            task.late_finish = max_finish
            task.late_start = task.late_finish - task.duration

    for task in sorted(self.tasks.values(), key=lambda t: -t.early_finish):
        for succ in task.successors:
            task.late_finish = min(task.late_finish, succ.late_start)
            task.late_start = task.late_finish - task.duration

def find_critical_path(self):
    return [task.name for task in self.tasks.values()
            if task.early_start == task.late_start]

def run(self):
    self.forward_pass()
```

```
    self.backward_pass()
    return self.find_critical_path()


tasks = {'A': 2, 'B': 3, 'C': 2, 'D': 4, 'E': 3, 'F': 2}

dependencies = { 'A': [], 'B': ['A'], 'C': ['A'], 'D': ['B'], 'E': ['B', 'C'], 'F': ['D', 'E'] }

cpm = CPM(tasks, dependencies)

critical_path = cpm.run()

print("Critical Path:", " -> ".join(critical_path))
```

OUTPUT

```
Critical Path: A -> B -> D -> F
```

RESULT:

Thus Critical Path Method (CPM) Analysis For Project Scheduling And Management Using Python Has Been Successfully Implemented.